

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Minería de Datos

Semestre I – 2026



Excelencia que trasciende

DELVALLE
GRUPO EDUCATIVO

Laboratorio 8

Integrantes:

Ciprian Jimenez, Mishell Rosa Elvira – 231169

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Ramírez Velásquez, Diego Alejandro – 23601

Enlace del repositorio: <https://github.com/Ninaswiftie09/Mineria-de-Datos/tree/Laboratorio8>

1. Preparación de los datos para los modelos SVM

Para esta parte del laboratorio se trabajó con la base `listings.RData`, correspondiente a propiedades de Airbnb. Inicialmente, el conjunto de datos cargado tenía 171,748 registros y 80 columnas. Después de preparar la variable de precio y eliminar registros sin precio válido, la base utilizada para el análisis quedó con 76,246 registros y 81 columnas. La columna adicional corresponde a la variable categórica `categoria_precio`, la cual clasifica las propiedades como baratas, medias o caras.

La variable `price` fue utilizada como base para construir la categoría de precio. Esta clasificación se hizo dividiendo los registros en tres grupos de tamaño similar, lo cual permitió que las clases quedaran bastante balanceadas. Esto es importante porque evita que el modelo aprenda más de una categoría solo porque tiene más ejemplos.

Cuadro 1: Resumen general del conjunto de datos

filas	columnas	precio	respuesta
76,246	81	<code>price</code>	<code>categoria_precio</code>

El conjunto de datos fue dividido en entrenamiento y prueba usando una semilla fija. Como no se encontraron archivos previos de entrenamiento y prueba dentro de la carpeta `data`, se generó una nueva división reproducible. El 70 % de los datos fue usado para entrenamiento y el 30 % para prueba.

Cuadro 2: División del conjunto de datos

conjunto	filas	columnas	origen del split
train	53,372	81	semilla
test	22,874	81	semilla

La distribución de la variable respuesta quedó similar entre las tres clases. Esto permite entrenar los modelos de clasificación sin necesidad de aplicar técnicas adicionales de balanceo.

Cuadro 3: Distribución de la variable respuesta

categoría	train	test
barata	17,982	7,707
media	17,607	7,546
cara	17,783	7,621

2. Exploración inicial de las variables

Al revisar la estructura del conjunto de datos, se observó que la base contiene variables numéricas, categóricas y de texto. La mayoría de las columnas son de tipo `object`, por lo que no podían usarse directamente en un modelo SVM. Por esta razón fue necesario aplicar varias transformaciones antes de entrenar el modelo.

Cuadro 4: Tipos de variables en la base

tipo	cantidad
object	46
int32	18
float64	16
category	1

También se revisaron los valores faltantes. Algunas columnas tenían demasiados valores vacíos, como `calendar_updated`, que no aportaba información, y otras columnas relacionadas con calificaciones tenían alrededor de 17.6 % de datos faltantes. En lugar de eliminar todos esos registros, se decidió imputar los valores faltantes durante el preprocesamiento.

En el caso de las variables numéricas, los valores faltantes se reemplazaron con la mediana. Para las variables categóricas, se utilizó la moda. Esta decisión permite conservar más registros y evita perder información útil para el entrenamiento.

3. Comportamiento de la variable precio

La variable `price` presentó una distribución bastante dispersa. El precio mínimo fue de 8, la mediana fue de 193 y el máximo llegó hasta 50,123. Esto muestra que existen valores extremos bastante altos, especialmente dentro de la categoría de propiedades caras.

Cuadro 5: Resumen del precio

count	mean	std	min	25 %	50 %	75 %	max
76,246	750.51	4,250.61	8	120	193	326	50,123

Al dividir los precios en categorías, se observa que las clases quedaron balanceadas en cantidad de registros. Sin embargo, la categoría cara tiene una media mucho mayor debido a la presencia de precios muy altos.

Cuadro 6: Precio por categoría

categoría	count	min	median	mean	max
barata	25,689	8	99	95.80	143
media	25,153	144	194	197.49	268
cara	25,404	269	437	1,960.12	50,123

Esto indica que la categoría de precio no solo separa propiedades por rangos, sino que también refleja diferencias importantes en el comportamiento del precio. En especial, la categoría cara tiene mayor variabilidad porque incluye propiedades con precios mucho más elevados que el resto.

4. Transformaciones realizadas para aplicar SVM

Antes de entrenar los modelos SVM fue necesario transformar la base, ya que este tipo de algoritmo trabaja mejor con variables numéricas y escaladas. Primero se eliminaron columnas que podían causar problemas, como identificadores, enlaces, imágenes, textos largos, columnas con demasiadas categorías y variables que podían representar fuga de información.

Entre las columnas eliminadas se encuentran identificadores, enlaces, columnas de texto largo, columnas con demasiados valores únicos y variables relacionadas directamente con ingresos. En el caso de `price` y `categoría_precio`, se eliminaron de las variables predictoras porque son la variable original y la variable respuesta, respectivamente. Mantenerlas dentro de `X` causaría fuga de información.

Las fechas como `last_scraped`, `host_since`, `first_review`, `last_review` y `calendar_last_scraped` fueron transformadas a variables numéricas en días. Esto permite conservar parte de la información temporal sin dejar las fechas como texto.

Después de limpiar las columnas, las variables numéricas fueron escaladas con estandarización, mientras que las variables categóricas fueron transformadas con One Hot Encoding. Este paso es necesario porque SVM es sensible a la escala de las variables y no acepta texto directamente como entrada.

Cuadro 7: Resumen de transformaciones aplicadas

parte	resultado
respuesta	categoria_precio
precio	price
split	semilla
faltantes numéricos	mediana
faltantes categóricos	moda
variables numéricas	escaladas
variables categóricas	One Hot Encoding
fechas transformadas	5
columnas eliminadas	30
matriz train	53,372 x 116
matriz test	22,874 x 116

Después del preprocesamiento, el conjunto de entrenamiento quedó representado por una matriz de 53,372 filas y 116 columnas, mientras que el conjunto de prueba quedó con 22,874 filas y 116 columnas. Esto significa que las variables originales fueron transformadas a un formato adecuado para entrenar modelos SVM.

5. Variable respuesta para clasificación

Para la construcción de los modelos de clasificación mediante Máquinas de Vectores de Soporte, se definió como variable respuesta la variable categórica `categoria_precio`. Esta variable clasifica las propiedades en tres niveles: barata, media y cara.

Esta transformación permite convertir el problema original de predicción de precios en un problema de clasificación multiclase, donde el objetivo es asignar correctamente cada propiedad a una de las categorías definidas. El uso de categorías facilita la interpretación de los resultados y permite identificar de forma más clara los distintos segmentos de mercado.

Cuadro 8: Variable respuesta utilizada en los modelos de clasificación

variable	tipo	categorías
<code>categoria_precio</code>	category	barata, cara, media

6. Modelos SVM para clasificación

Se implementaron distintos modelos de Máquinas de Vectores de Soporte utilizando kernel lineal, radial o RBF y polinomial. Para cada modelo se ajustaron valores de C , γ y degree ,

según correspondiera. Debido al costo computacional de SVM, se utilizó una muestra fija del conjunto de entrenamiento y una muestra fija del conjunto de prueba. Todos los modelos se compararon bajo las mismas condiciones.

Cuadro 9: Muestras utilizadas para clasificación

conjunto	filas usadas	columnas antes de codificar
train	5,000	56
test	2,000	56

Cuadro 10: Resultados de los modelos SVM evaluados

modelo	acc. train	acc. test	precisión	recall	F1	errores
SVM RBF C=1 gamma=scale	0.771	0.698	0.697	0.698	0.697	605
SVM RBF C=10 gamma=scale	0.900	0.698	0.696	0.698	0.696	604
SVM polinomial C=1 d=2	0.742	0.691	0.696	0.691	0.693	619
SVM RBF C=1 gamma=0.01	0.732	0.690	0.688	0.690	0.688	621
SVM polinomial C=1 d=3	0.758	0.679	0.698	0.679	0.683	642
SVM lineal C=1	0.690	0.678	0.674	0.678	0.675	643
SVM lineal C=0.1	0.686	0.676	0.671	0.676	0.672	647

El mejor modelo SVM según F1-score ponderado fue el SVM RBF con C=1 y gamma=scale. Este modelo alcanzó un accuracy de prueba de 0.6975 y un F1-score de 0.6969. Aunque el SVM RBF con C=10 tuvo un accuracy casi igual, su diferencia entre entrenamiento y prueba fue mayor, por lo que el modelo con C=1 muestra un comportamiento más equilibrado.

7. Predicción de la variable respuesta

Después de entrenar los modelos, se utilizaron para predecir la categoría de precio en el conjunto de prueba. La siguiente tabla muestra una muestra de los valores reales y predichos por el mejor modelo SVM.

Cuadro 11: Ejemplo de predicciones del mejor modelo SVM

observación	valor real	predicción
1	cara	cara
2	barata	media
3	media	barata
4	media	media
5	cara	cara
6	cara	cara
7	media	media
8	barata	barata
9	media	media
10	cara	cara

Este resultado muestra que el modelo logra clasificar correctamente varias observaciones, pero también confunde algunos casos de la categoría barata y media. Esto es esperable porque los límites entre categorías cercanas pueden compartir características similares.

8. Matrices de confusión y evaluación por clase

Para evaluar el desempeño de los modelos, se utilizaron matrices de confusión y métricas de clasificación como precisión, recall y F1-score. Estas métricas permiten analizar el comportamiento del modelo en cada una de las categorías de precio y no solo ver el porcentaje global de aciertos.

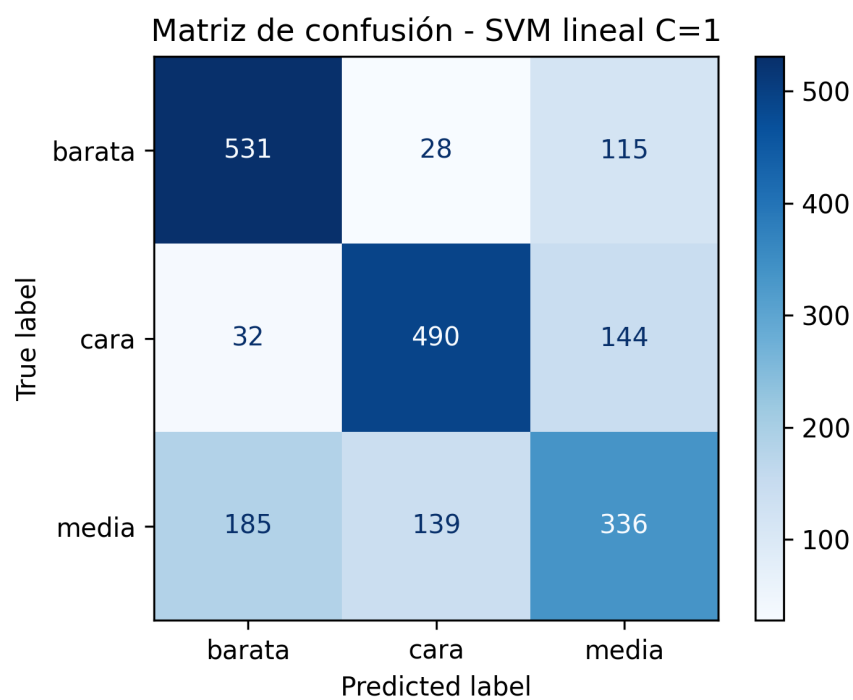


Figura 1: Matriz de confusión del modelo SVM lineal. Archivo: grafico1.png

Cuadro 12: Reporte de clasificación del modelo SVM lineal

clase	precisión	recall	F1-score	support
barata	0.710	0.788	0.747	674
cara	0.746	0.736	0.741	666
media	0.565	0.509	0.535	660
accuracy	0.678	0.678	0.678	2000
macro avg	0.673	0.678	0.674	2000
weighted avg	0.674	0.678	0.675	2000

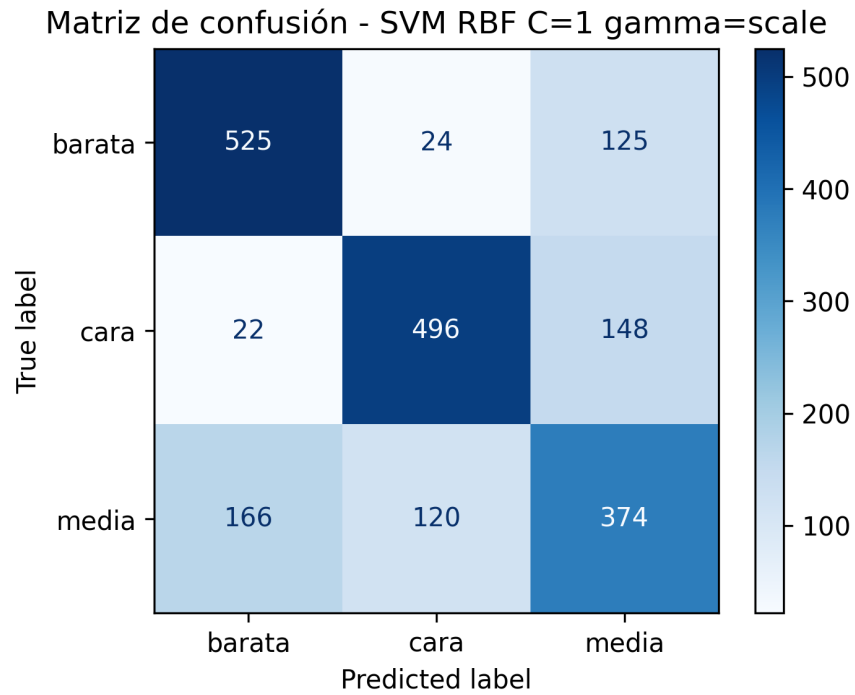


Figura 2: Matriz de confusión del modelo SVM RBF. Archivo: grafico2.png

Cuadro 13: Reporte de clasificación del modelo SVM RBF

clase	precisión	recall	F1-score	support
barata	0.736	0.779	0.757	674
cara	0.775	0.745	0.760	666
media	0.578	0.567	0.572	660
accuracy	0.698	0.698	0.698	2000
macro avg	0.696	0.697	0.696	2000
weighted avg	0.697	0.698	0.697	2000

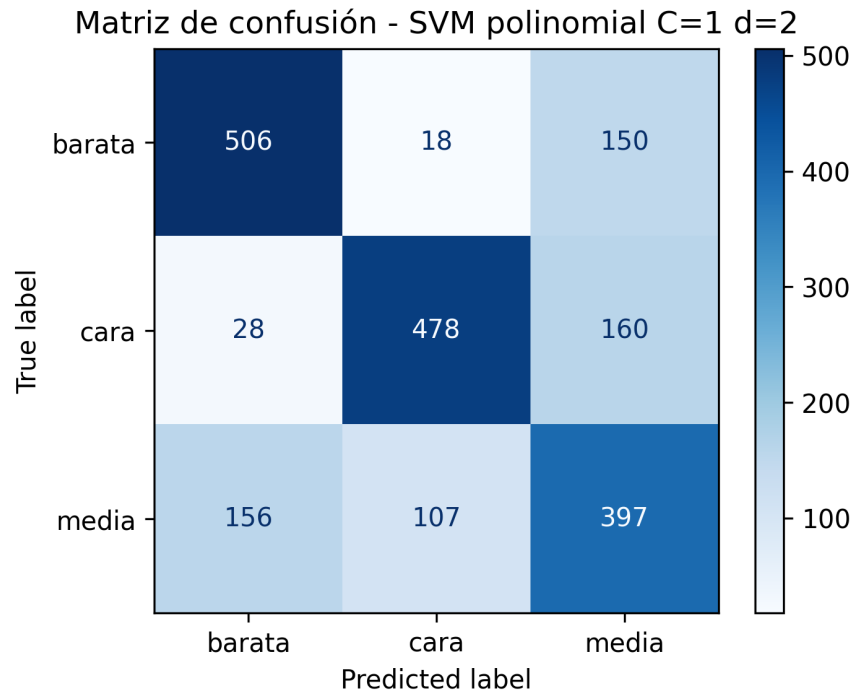


Figura 3: Matriz de confusión del modelo SVM polinomial. Archivo: grafico3.png

Cuadro 14: Reporte de clasificación del modelo SVM polinomial

clase	precisión	recall	F1-score	support
barata	0.733	0.751	0.742	674
cara	0.793	0.718	0.753	666
media	0.562	0.602	0.581	660
accuracy	0.690	0.690	0.690	2000
macro avg	0.696	0.690	0.692	2000
weighted avg	0.696	0.690	0.693	2000

El modelo RBF obtuvo el mejor desempeño general, alcanzando un accuracy cercano a 0.70. Las categorías barata y cara presentan mejores resultados, mientras que la categoría media es la más difícil de clasificar. Esto puede deberse a que las propiedades de precio medio se encuentran entre los rangos bajo y alto, por lo que sus características se solapan más con las otras clases.

9. Errores principales del mejor modelo SVM

El análisis de errores permite observar en qué casos el modelo se equivoca con mayor frecuencia. Esto es importante porque no todos los errores tienen el mismo impacto para una empresa como SmartStay. Por ejemplo, clasificar una propiedad media como barata podría afectar la estrategia de recomendación y el ingreso esperado.

Cuadro 15: Principales confusiones del mejor modelo SVM

real	predicho	cantidad
media	barata	166
cara	media	148
barata	media	125
media	cara	120
barata	cara	24
cara	barata	22

El error más frecuente del mejor modelo fue clasificar propiedades reales de categoría media como barata, con 166 casos. Esto confirma que la categoría media es la más compleja de separar, ya que comparte características con propiedades baratas y caras.

10. Análisis de sobreajuste en los modelos SVM

Para analizar el sobreajuste se comparó el desempeño del modelo en entrenamiento contra el desempeño en prueba. Si un modelo obtiene un resultado muy alto en entrenamiento, pero baja mucho en prueba, entonces probablemente está memorizando los datos y no generaliza bien.

Cuadro 16: Análisis de sobreajuste en modelos SVM

modelo	acc. train	acc. test	diferencia	diagnóstico
SVM RBF C=1 gamma=scale	0.771	0.698	0.074	ajuste aceptable
SVM RBF C=10 gamma=scale	0.900	0.698	0.202	posible sobreajuste
SVM polinomial C=1 d=2	0.742	0.691	0.052	ajuste aceptable
SVM RBF C=1 gamma=0.01	0.732	0.690	0.043	ajuste aceptable
SVM polinomial C=1 d=3	0.758	0.679	0.079	ajuste aceptable
SVM lineal C=1	0.690	0.678	0.011	ajuste aceptable
SVM lineal C=0.1	0.686	0.676	0.009	ajuste aceptable

En general, la mayoría de modelos SVM no muestran una diferencia demasiado grande entre entrenamiento y prueba. Sin embargo, el modelo RBF con C=10 sí presenta una diferencia de 0.202, por lo que se considera un posible caso de sobreajuste. Para manejar este problema se pueden probar valores más bajos de C, reducir la complejidad del kernel, usar validación cruzada, aumentar la cantidad de datos de entrenamiento o seleccionar variables más relevantes.

11. Comparación con otros modelos de clasificación

Para comparar el mejor modelo SVM con los algoritmos trabajados anteriormente, se entrenaron modelos de referencia usando la misma muestra de entrenamiento, la misma muestra de prueba y el mismo preprocesamiento. Esto permite comparar de forma más justa el desempeño de árboles de decisión, Random Forest, Bayes Ingenuo, KNN, Regresión Logística y SVM.

Cuadro 17: Comparación del mejor modelo por algoritmo de clasificación

algoritmo	modelo	acc. train	acc. test	precisión	F1	tiempo
Random Forest	Random Forest 80 árboles	1.000	0.709	0.707	0.707	0.416
SVM	SVM RBF C=1 gamma=scale	0.771	0.698	0.697	0.697	1.308
Regresión logística	Regresión logística C=1	0.683	0.682	0.678	0.679	0.610
Árbol de decisión	Árbol de decisión max_depth=10	0.807	0.662	0.662	0.661	0.206
KNN	KNN k=11	0.704	0.627	0.626	0.624	0.100
Naive Bayes	Naive Bayes Gaussiano	0.524	0.505	0.591	0.470	0.117

El mejor modelo de clasificación fue Random Forest, con un accuracy de prueba de 0.709 y un F1-score de 0.707. El modelo SVM RBF quedó muy cerca, con un accuracy de 0.698. Aunque SVM no fue el mejor en esta comparación, sí ofrece un desempeño competitivo y supera a regresión logística, árbol de decisión, KNN y Naive Bayes.

12. Comparación del sobreajuste en clasificación

Para revisar el sobreajuste de todos los modelos de clasificación, se comparó el accuracy de entrenamiento con el accuracy de prueba. La diferencia entre ambas métricas permite observar qué modelos generalizan mejor y cuáles se ajustan demasiado a los datos de entrenamiento.

Cuadro 18: Comparación del sobreajuste en clasificación

algoritmo	modelo	acc. train	acc. test	diferencia	diagnóstico
Random Forest	Random Forest 80 árboles	1.000	0.709	0.292	posible sobreajuste
SVM	SVM RBF C=1 gamma=scale	0.771	0.698	0.074	ajuste aceptable
Regresión logística	Regresión logística C=1	0.683	0.682	0.000	ajuste aceptable
Árbol de decisión	Árbol de decisión max_depth=10	0.807	0.662	0.145	ajuste aceptable
KNN	KNN k=11	0.704	0.627	0.076	ajuste aceptable
Naive Bayes	Naive Bayes Gaussiano	0.524	0.505	0.019	posible desajuste

El modelo con mayor diferencia entre entrenamiento y prueba fue Random Forest, por lo que muestra la mayor tendencia al sobreajuste dentro de esta comparación. En cambio, regresión logística casi no presenta diferencia entre entrenamiento y prueba, aunque su desempeño es menor que Random Forest y SVM. Naive Bayes muestra señales de desajuste porque su accuracy de prueba es bajo.

13. Modelo de regresión con SVR

Después de evaluar la clasificación de categorías de precio, se construyó un modelo de regresión para predecir directamente el precio de las propiedades. En este caso se utiliza `price` como variable respuesta. Debido a que el precio tiene valores extremos, se aplicó una transformación logarítmica interna al objetivo durante el entrenamiento y luego se regresó la predicción a la escala original.

Cuadro 19: Muestras utilizadas para regresión

conjunto	filas usadas	columnas antes de codificar
train	3,000	56
test	1,500	56

Cuadro 20: Resultados de los modelos SVR evaluados

modelo	MAE train	MAE test	RMSE train	RMSE test	R^2 train	R^2 test
SVR RBF C=100 epsilon=0.2	118.271	300.987	697.255	2,306.934	0.967	0.572
SVR RBF C=10 epsilon=0.1	186.369	324.458	2,134.361	2,716.349	0.686	0.406
SVR polinomial C=10 d=2	302.444	378.613	2,686.493	2,955.770	0.503	0.297
SVR lineal C=1 epsilon=0.1	462.099	418.350	3,447.138	3,300.018	0.181	0.124

El mejor modelo SVR fue el SVR RBF con C=100 y epsilon=0.2, ya que obtuvo el menor RMSE de prueba, con 2,306.934. También alcanzó el mejor R^2 de prueba dentro de los modelos SVR evaluados.

14. Comparación de modelos de regresión

Para comparar el modelo SVR con algoritmos de regresión usados en entregas anteriores, se entrenaron modelos de referencia bajo las mismas condiciones. En esta parte se comparan Regresión Lineal, Árbol de Regresión, Random Forest, KNN y SVR. Bayes Ingenuo no se incluye como regresor directo porque su formulación estándar se utiliza para clasificación, no para predecir una variable continua como el precio.

Cuadro 21: Comparación del mejor modelo por algoritmo de regresión

algoritmo	modelo	MAE test	RMSE train	RMSE test	R^2 test	tiempo
SVR	SVR RBF C=100 epsilon=0.2	300.987	697.255	2,306.934	0.572	1.420
KNN	KNN k=5	330.470	3,039.232	2,641.583	0.438	0.091
Random Forest	Random Forest 80 árboles	360.519	2,090.631	2,868.402	0.338	1.465
Regresión lineal	Regresión lineal	438.377	3,434.992	3,402.621	0.068	0.146
Árbol de regresión	Árbol de regresión max_depth=14	533.334	77.641	3,893.767	-0.220	0.204
Naive Bayes	No aplica para regresión continua	–	–	–	–	–

El mejor modelo de regresión fue SVR RBF con C=100 y epsilon=0.2, porque obtuvo el menor RMSE de prueba. En regresión, el RMSE es una métrica útil porque penaliza más los errores grandes, lo cual es importante cuando se predicen precios con valores extremos.

15. Análisis de sobreajuste en regresión

En regresión también se puede revisar el sobreajuste comparando el desempeño en entrenamiento y prueba. En este caso se comparan principalmente el RMSE y el R^2 . Si el modelo tiene un R^2 muy alto en entrenamiento, pero mucho más bajo en prueba, o si el RMSE aumenta mucho en prueba, entonces puede estar sobreajustado.

Cuadro 22: Comparación del sobreajuste en regresión

algoritmo	modelo	RMSE train	RMSE test	R^2 train	R^2 test	diagnóstico
SVR	SVR RBF C=100 epsilon=0.2	697.255	2,306.934	0.967	0.572	posible sobreajuste
KNN	KNN k=5	3,039.232	2,641.583	0.364	0.438	ajuste aceptable
Random Forest	Random Forest 80 árboles	2,090.631	2,868.402	0.699	0.338	posible sobreajuste
Regresión lineal	Regresión lineal	3,434.992	3,402.621	0.187	0.068	ajuste aceptable
Árbol de regresión	Árbol de regresión max_depth=14	77.641	3,893.767	1.000	-0.220	posible sobreajuste

El SVR fue el mejor modelo de regresión en prueba, pero también muestra una diferencia importante entre entrenamiento y prueba. Esto indica que, aunque predice mejor que los otros modelos comparados, podría estar ajustándose demasiado al conjunto de entrenamiento. El árbol de regresión presenta el caso más fuerte de sobreajuste, porque logra un resultado casi perfecto en entrenamiento, pero su desempeño cae bastante en prueba.

16. Conclusiones generales

Los modelos SVM permitieron clasificar las propiedades en categorías de precio con un desempeño aceptable. El mejor modelo SVM fue el RBF con C=1 y gamma=scale, ya que tuvo el mejor F1-score ponderado y mantuvo una diferencia razonable entre entrenamiento y prueba.

En las matrices de confusión se observó que la categoría media fue la más difícil de clasificar. Esto tiene sentido porque las propiedades de precio medio comparten características tanto con

propiedades baratas como con propiedades caras. Por eso, varias equivocaciones del modelo ocurrieron entre la categoría media y las otras dos clases.

Al comparar SVM con otros modelos de clasificación, Random Forest obtuvo el mejor accuracy, aunque también presentó mayor tendencia al sobreajuste. SVM quedó muy cerca en desempeño y mostró un ajuste más equilibrado. Esto lo convierte en una alternativa útil cuando se busca un balance entre precisión y generalización.

En regresión, el mejor resultado lo obtuvo el modelo SVR RBF con $C=100$ y $\epsilon=0.2$. Este modelo consiguió el menor RMSE de prueba, por lo que fue el más adecuado para estimar directamente el precio de las propiedades. Sin embargo, también presentó señales de sobreajuste, por lo que sería conveniente seguir ajustando hiperparámetros o probar validación cruzada.

En general, el análisis muestra que no basta con elegir el modelo que tiene mejor resultado en entrenamiento. Para seleccionar un modelo útil para SmartStay, es necesario revisar su desempeño en prueba, las métricas por clase, las matrices de confusión, el tiempo de procesamiento y el posible sobreajuste.